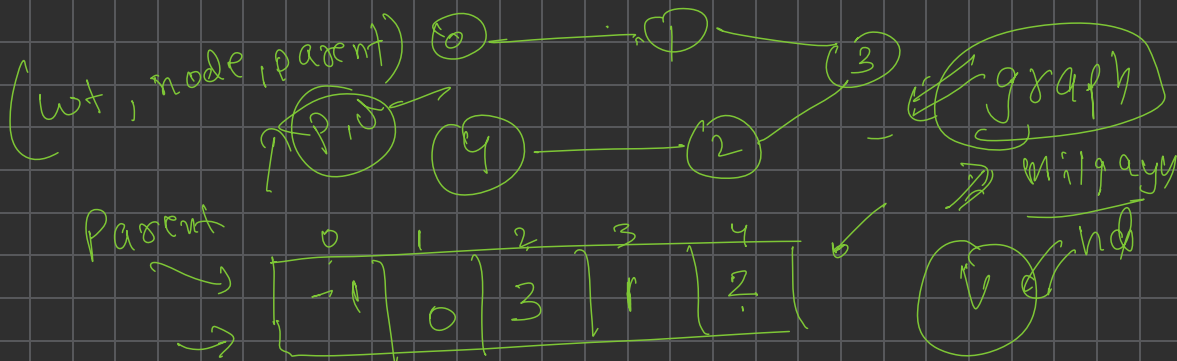
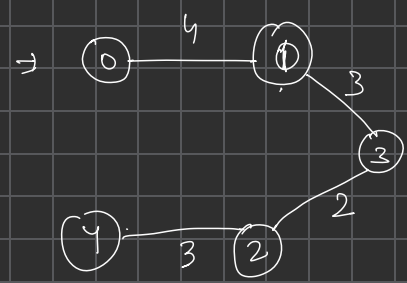
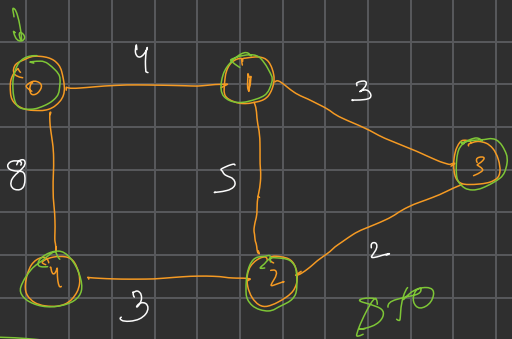






① → Prim Algo! → Path exist

Cost = 8
7
9 12



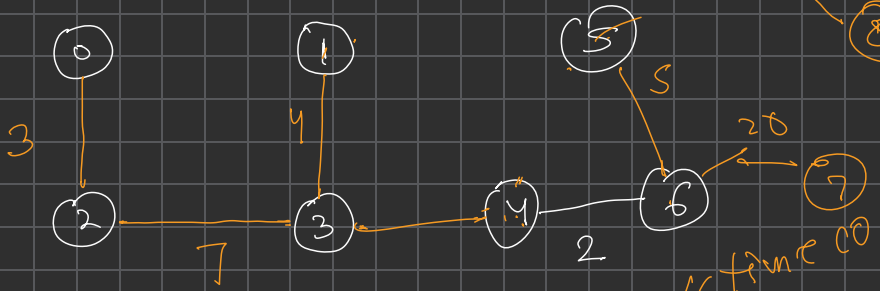
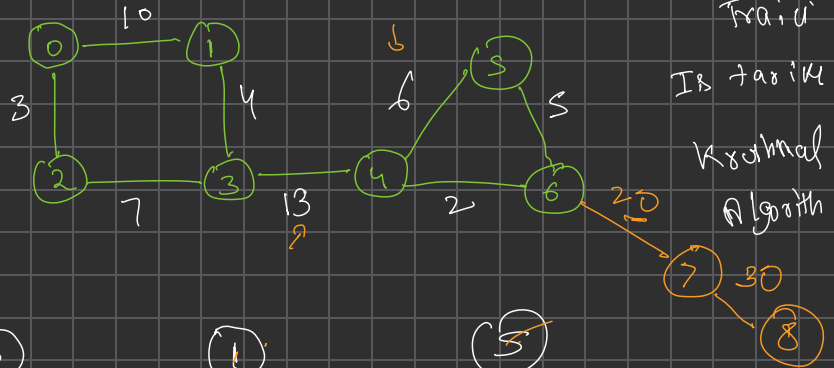
8, 4, 0
5, 2, 1

node =
wt =
parent =

Magical Method $\Rightarrow O(1)$

Kruskal Algo ✓

Minimum Span Tree
Is a tree
Kruskal Algorithm



Array

0	1	2	3	4	5	6	7	8
0	0	0	0	0	4	0	0	0

0 → 8
all

$(5, 8)$ ✓

$(0, 2)$

$(2, 8) \rightarrow (0, 3) \Rightarrow \checkmark$ 1198

$\rightarrow (0, 1)$
 $\rightarrow (1, 2)$
 $\rightarrow (3, 4)$
 $\rightarrow (5, 6)$
 $\rightarrow (7, 8)$
 $\rightarrow (3, 6)$

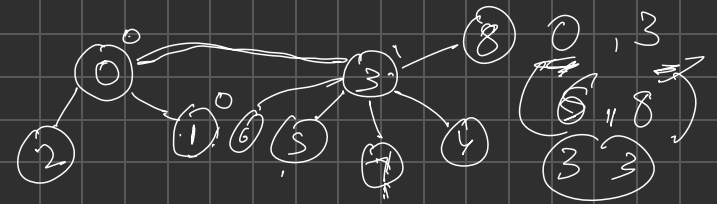
Parent
801111

0	1	2	3	4	5	6	7	8
3	0	0	3	3	3	3	3	3
1	0	0	2	0	1	0	1	0

(0) (1) (2) (3) (4) (5) (6) (7) (8)

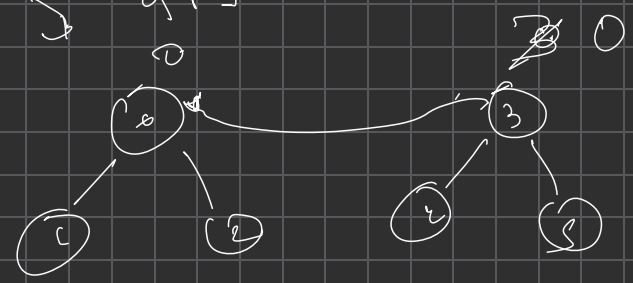
$(3, 7)$

① First find the ultimate parent of both 3 & 7

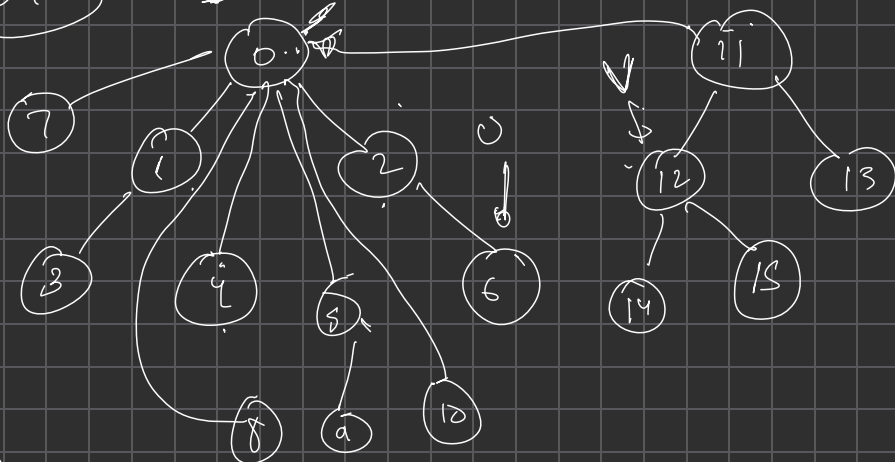


② If both have same parent then first and second rank of RM

(2)



$(6, 11) \Rightarrow \text{same comp}$
 $(7, 10)$
 $(0, 10)$
 $\Rightarrow \text{rank}$
 $V1$
 $\underline{\underline{3}}$



$(2, 10)$

$\downarrow$
 $\{$ int findParent(int u, vector<int> parent;
 if (u == parent[u])
 return u;

return parent[u] = findParent(parent[u], parent)

y

```
void findUnion(int u, int v, vector<int> parent, rank)
```

```
{  
    int pu = findParent(u, parent)
```

```
    int pv = findParent(v, parent)
```

```
    if (rank[pu] > rank[pv])
```

```
        parent[pv] = pu;
```

```
    else if (rank[pu] < rank[pv])
```

```
        parent[pu] = pv;
```

```
    else
```

```
    {
```

```
        rank[pu]++;
```

```
        parent[pu] = pv;
```

```
    }
```

```
}
```

